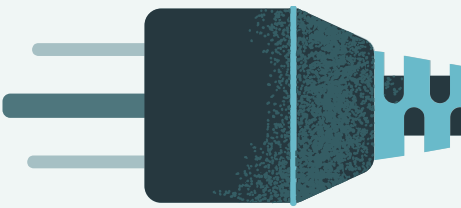
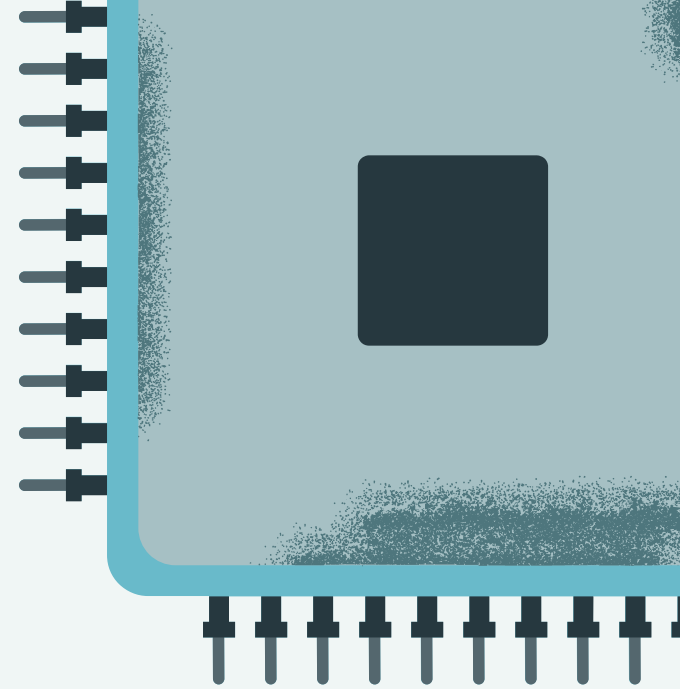
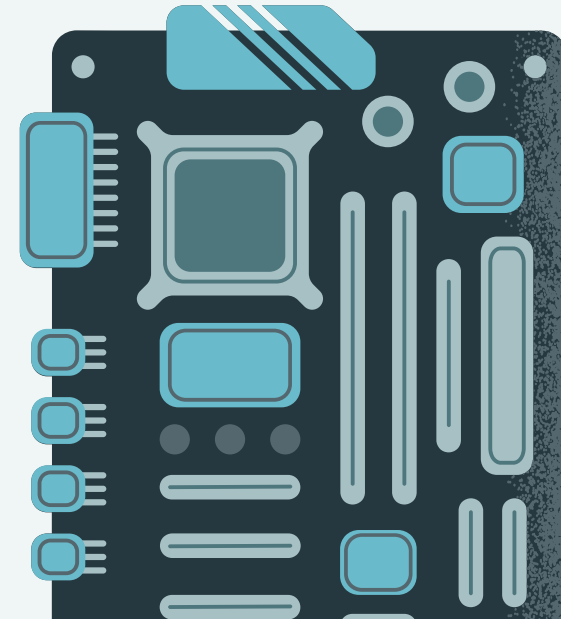
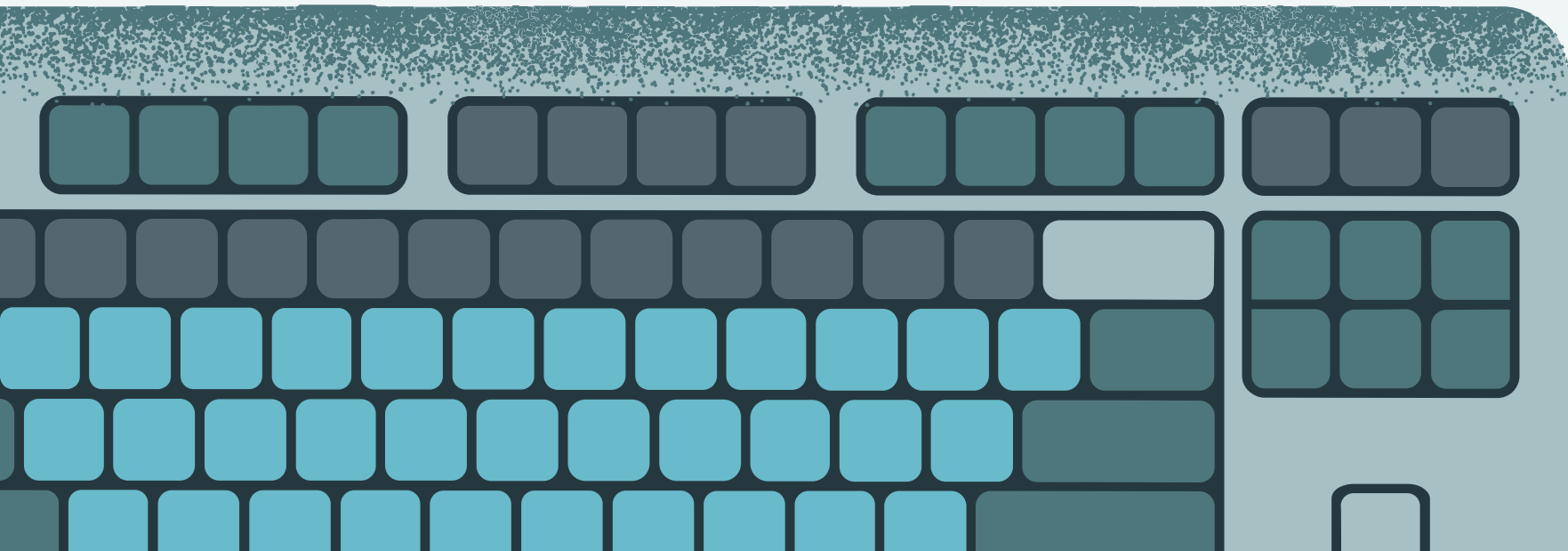
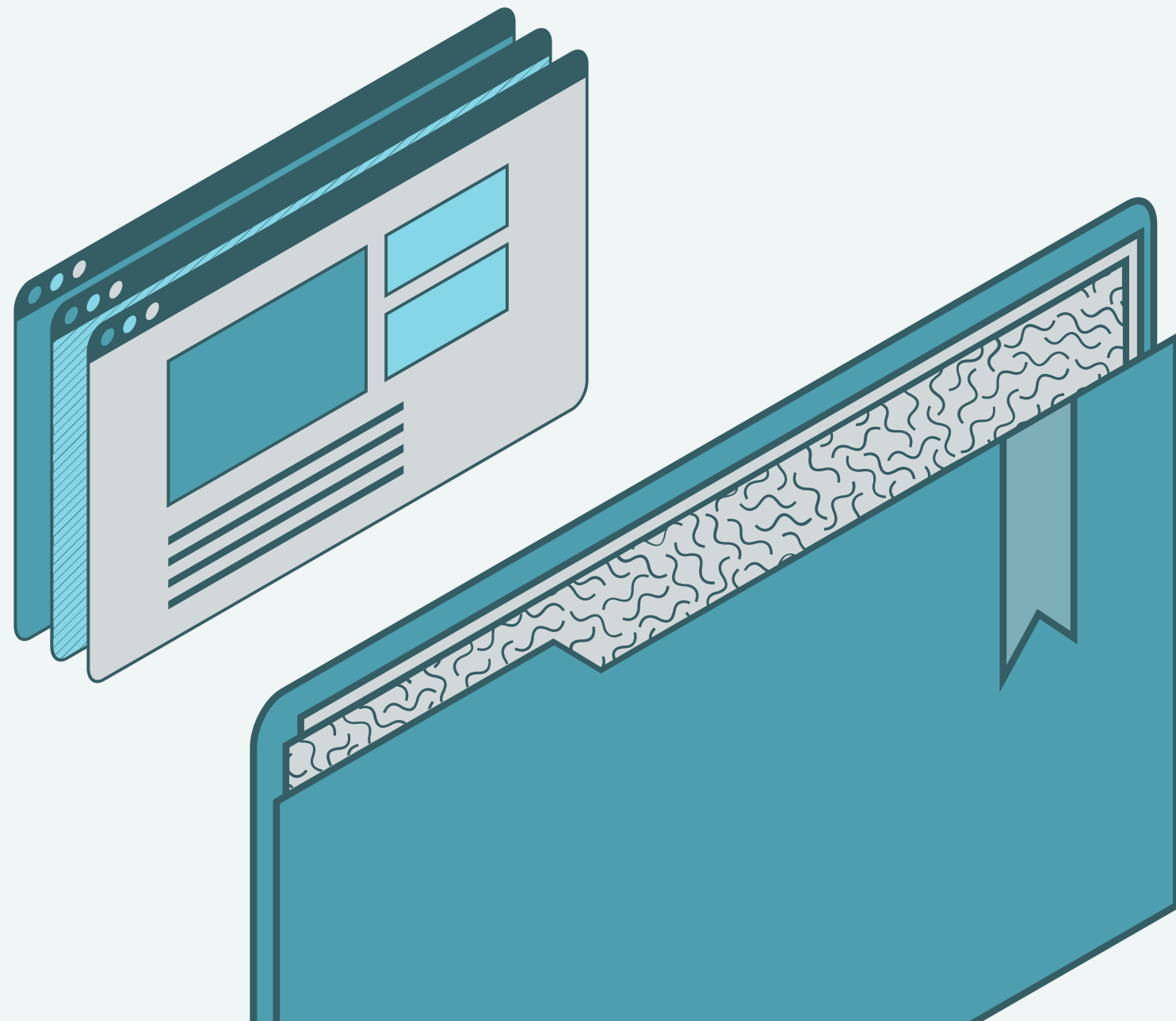


Equipo 5

ALGORITMOS DIVIDE Y VENCERAS



ÍNDICE



01. Introducción

02. Características

03. Ventajas y Desventajas

04. Uso en Ciencia y Tecnología

05. Búsqueda Binaria

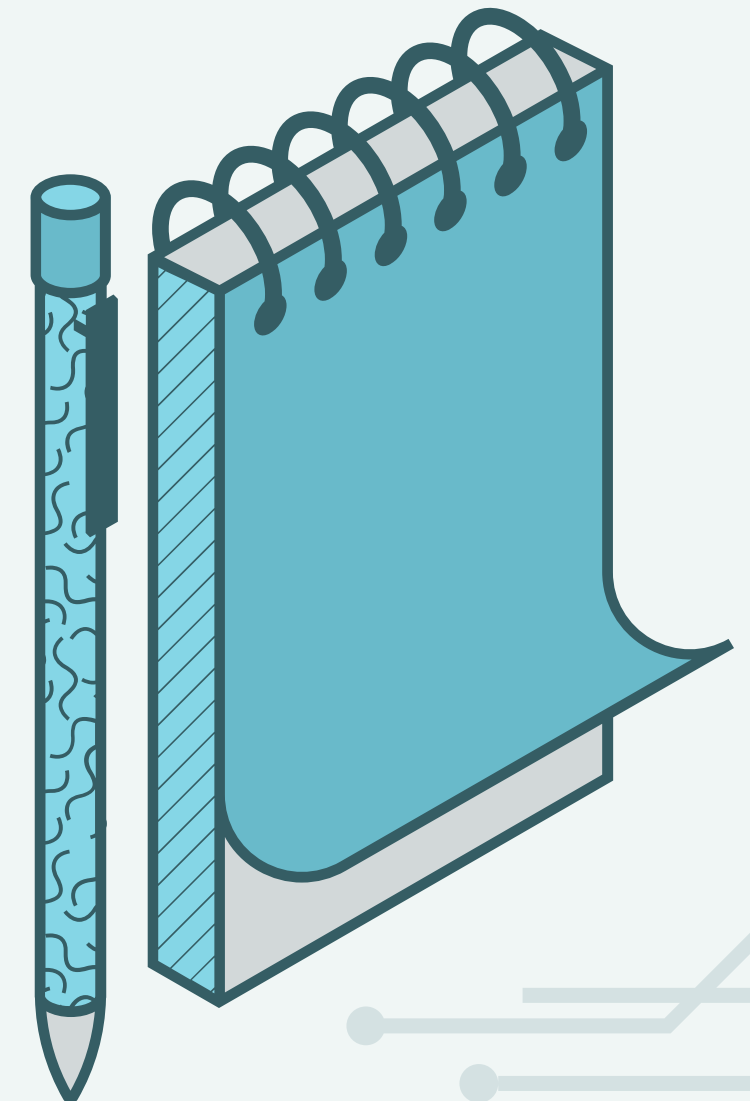
06. Quick Sort

07. Merge Sort

08. Actividad

INTRODUCCIÓN

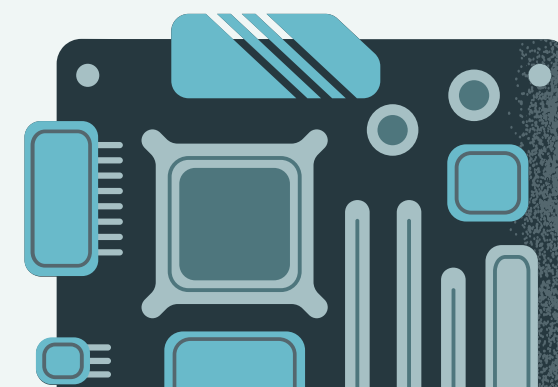
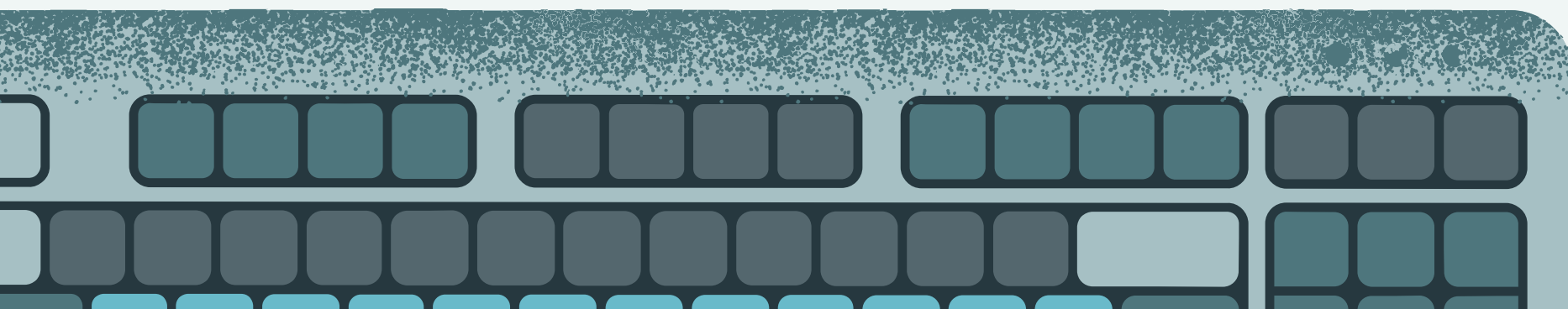
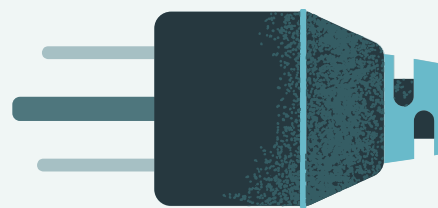
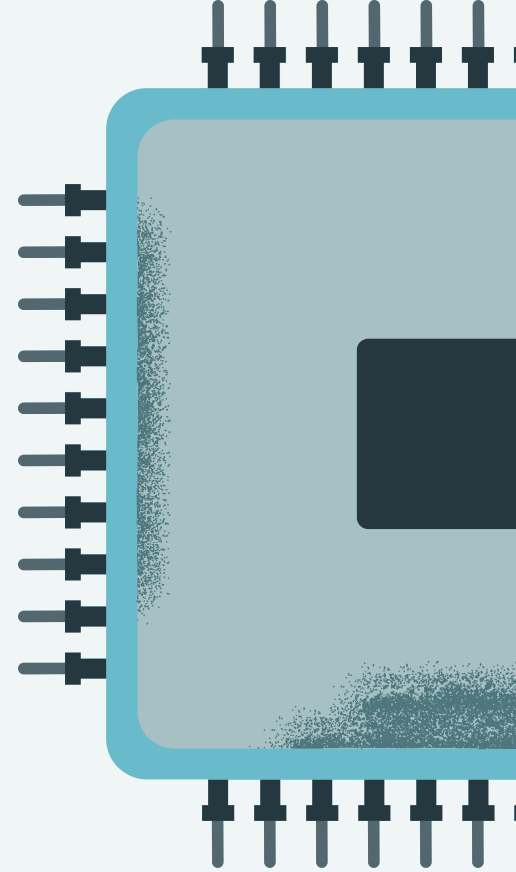
El paradigma de **Divide y Vencerás (Divide and Conquer)** es una de las técnicas más utilizadas en el diseño de algoritmos. Su idea principal es dividir un problema complejo en subproblemas más pequeños y manejables, resolver cada uno de ellos (recursivamente, en muchos casos) y finalmente combinar las soluciones para obtener el resultado final.



ACERCA DE

En C++, estos algoritmos se implementan naturalmente mediante funciones recursivas, aprovechando la pila de llamadas del sistema para manejar los subproblemas de forma automática.

La mayoría de los algoritmos clásicos más eficientes — Quick Sort, Merge Sort, búsqueda binaria, multiplicación de matrices de Strassen — son implementaciones directas de este paradigma.



CARACTERÍSTICAS

1

DESCOMPOSICIÓN RECURSIVA

- Dividir el problemas en subproblemas mas pequeños del mismo tipo.

2

CASO BASE

- Detener la recursion cuando se cumple cierta condición.

3

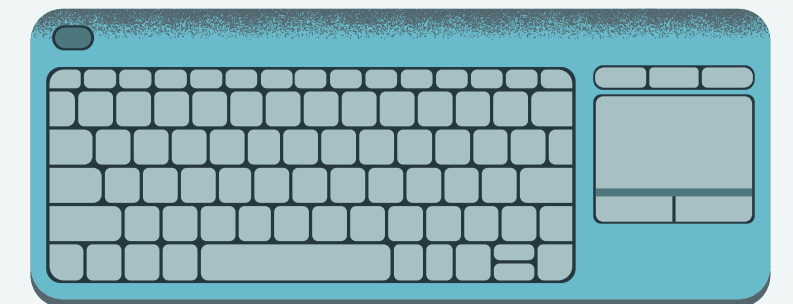
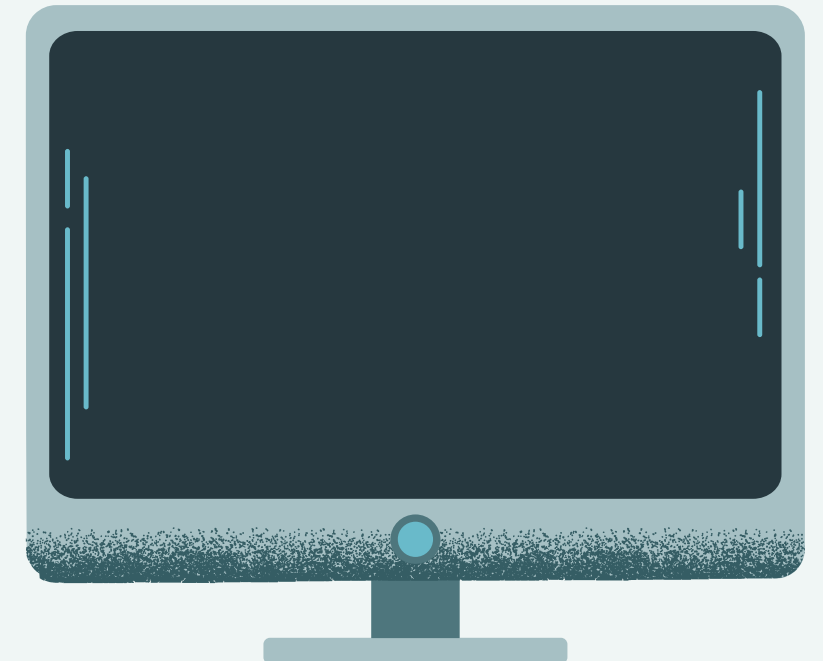
COMBINACIÓN DE SOLUCIONES

- Una vez resueltos los subproblemas, integrar las soluciones .

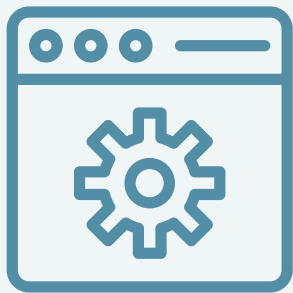
4

PARALELIZABLE

- Los subproblemas independientes pueden resolverse en paralelo con múltiples hilos.



VENTAJAS



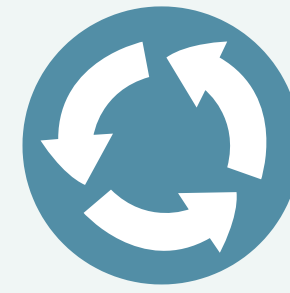
EFICIENCIA

Los algoritmos de divide y vencerás suelen proporcionar soluciones eficientes a los problemas.



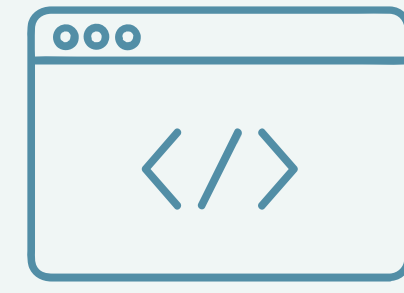
MODULARIDAD

El algoritmo promueve la modularidad dividiendo un problema complejo en subproblemas manejables.



REUTILIZACIÓN

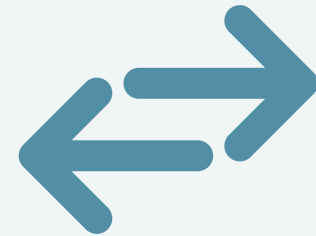
Una vez diseñado un algoritmo general de divide y vencerás, puede aplicarse a problemas con características similares.



OPTIMALIDAD

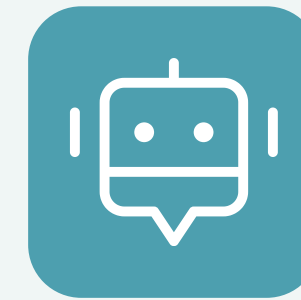
Al resolver los subproblemas de forma óptima y combinar correctamente los resultados, el algoritmo puede garantizar una solución óptima para el problema global.

VENTAJAS



PARALELIZACIÓN

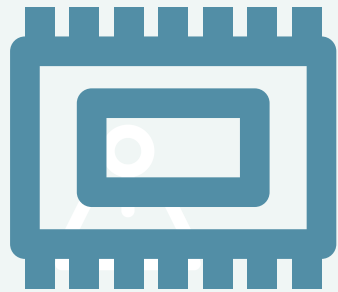
Se utilizan en máquinas multiprocesador con sistemas de memoria compartida donde la comunicación de datos entre procesadores no necesita planificarse porque se pueden ejecutar subproblemas distintos en diferentes procesadores.



RESOLVER GRANDES INSTANCIAS

Al dividir el problema en subproblemas más pequeños, el algoritmo puede manejar entradas más grandes de forma más eficiente.

DESVENTAJAS



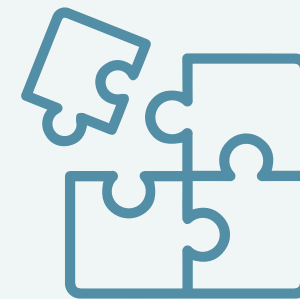
USO EXTRA DE MEMORIA

En algunos casos, el enfoque de dividir y vencer requiere memoria adicional para almacenar resultados intermedios durante el proceso recursivo.



DIFICULTAD EN CIERTOS PROBLEMAS

No todos los problemas pueden resolverse fácilmente usando el paradigma de divide y vencerás.



SOLUCIONES SUBÓPTIMAS

Hay casos en los que algoritmos alternativos pueden superar a los enfoques de divide y vencerás en términos de complejidad temporal o espacial.



NO SIEMPRE ESTABLE

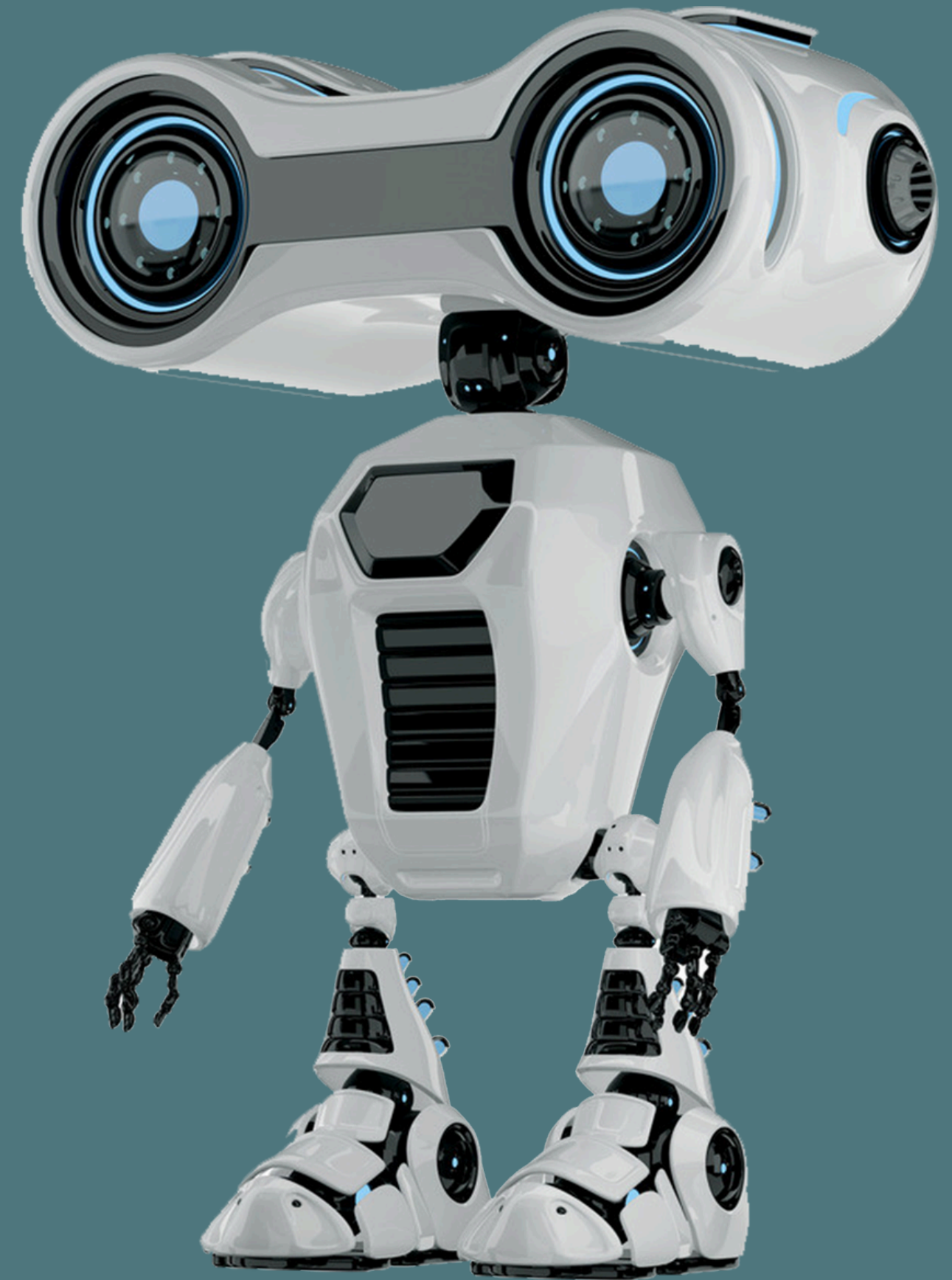
Los algoritmos de divide y vencerás pueden no preservar el orden de los elementos en la entrada.

DONDE NO USARLO

La técnica de divide y vencerás no siempre es la mejor opción para resolver todos los problemas. Por ejemplo:

- Cuando el problema es demasiado complejo y no se puede dividir en subproblemas manejables.
- Casos en los que los subproblemas no son independientes entre sí.
- Restricciones que no pueden resolverse de manera recursiva.
- Problemas conmutativos donde el orden de resolución no afecta el resultado.
- Donde genere costos adicionales de tiempo o recursos que no compensen sus beneficios.

Por esta razón, es importante analizar las características de cada problema antes de decidir si conviene utilizar la estrategia de divide y vencerás.

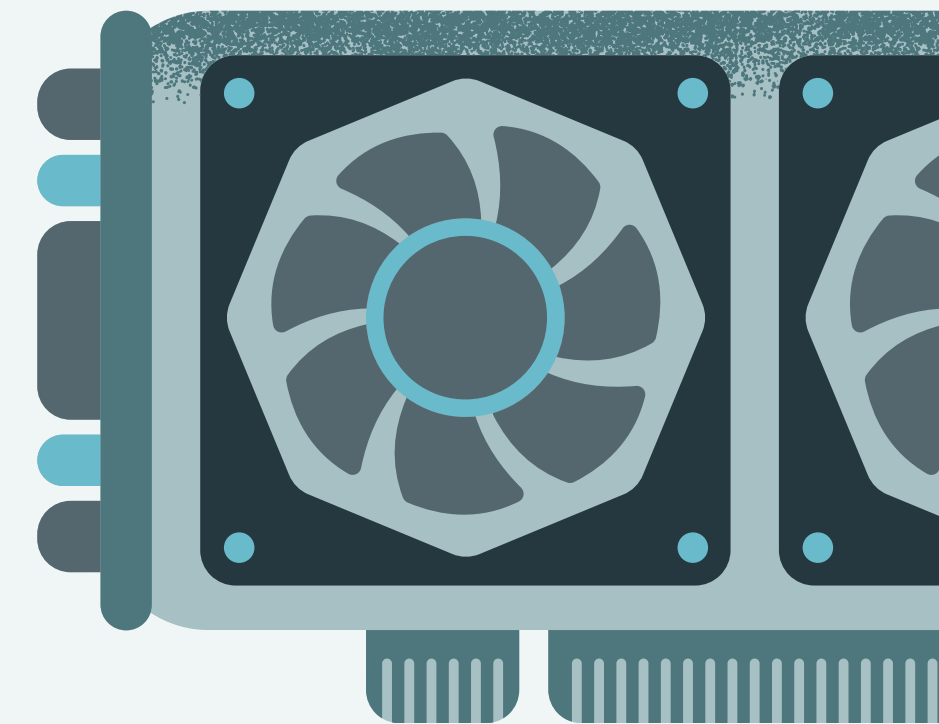


USO EN CIENCIA Y TECNOLOGÍA

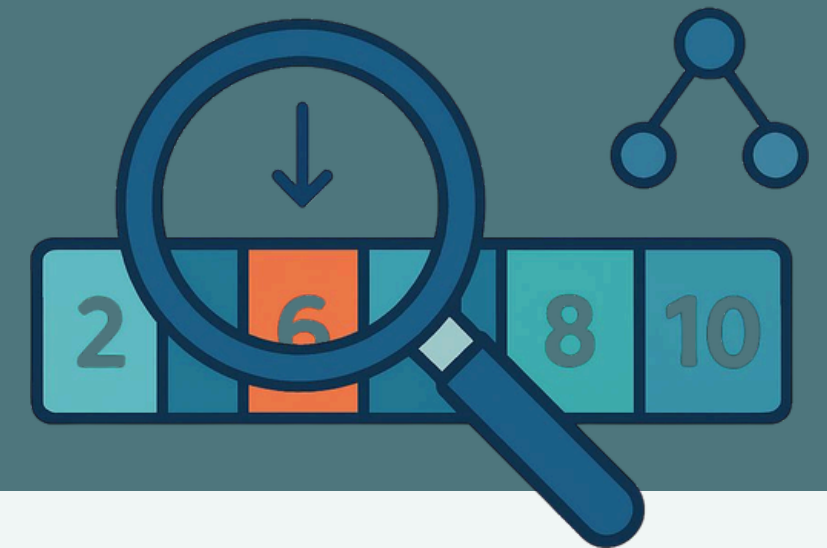


La estrategia de divide y vencerás se utiliza mucho en informática, matemáticas aplicadas, ingeniería, tecnología, algoritmos, e inteligencia artificial .

- Ordenación de datos
- Búsqueda
- Multiplicación de números grandes
- Transformada rápida de Fourier
FFT
- Resolución de ecuaciones
diferenciables
- Teoría de grafos
- Enrutamiento y análisis de tráfico
en la red
- Diseño de compiladores
- Árboles de decisiones
- Algoritmos de clustering
- Optimización de modelos



BUSQUEDA BINARIA

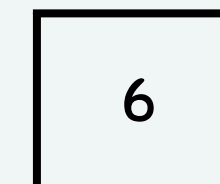
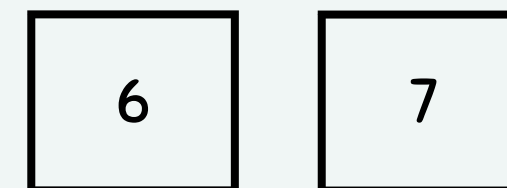
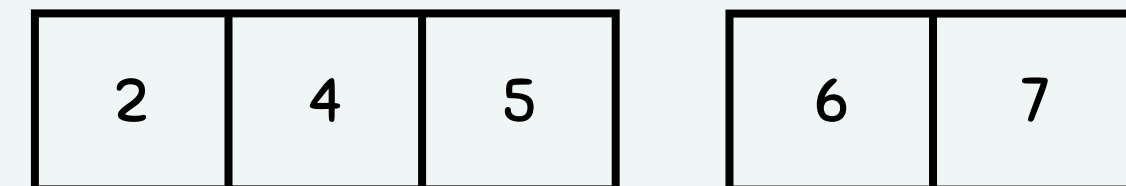
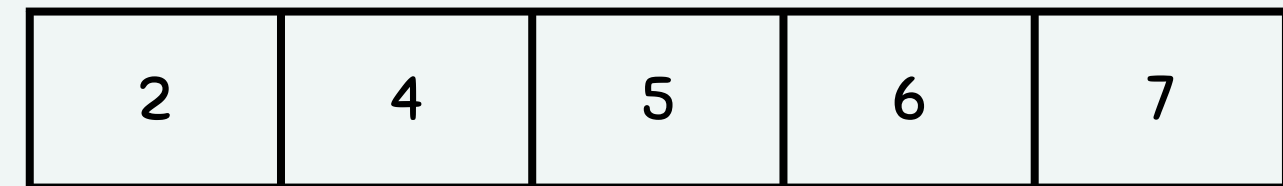


Es un algoritmo que permite encontrar un elemento en una colección ordenada reduciendo el rango de búsqueda a la mitad en cada paso.

Ejemplo Visual:

Num a buscar: 6

- **Dividir:** se toma el elemento central.
- **Vencer:** se decide en qué mitad continuar y se descarta la otra.
- **Complejidad:** $O(\log n)$, mucho más eficiente que la búsqueda lineal $O(n)$.

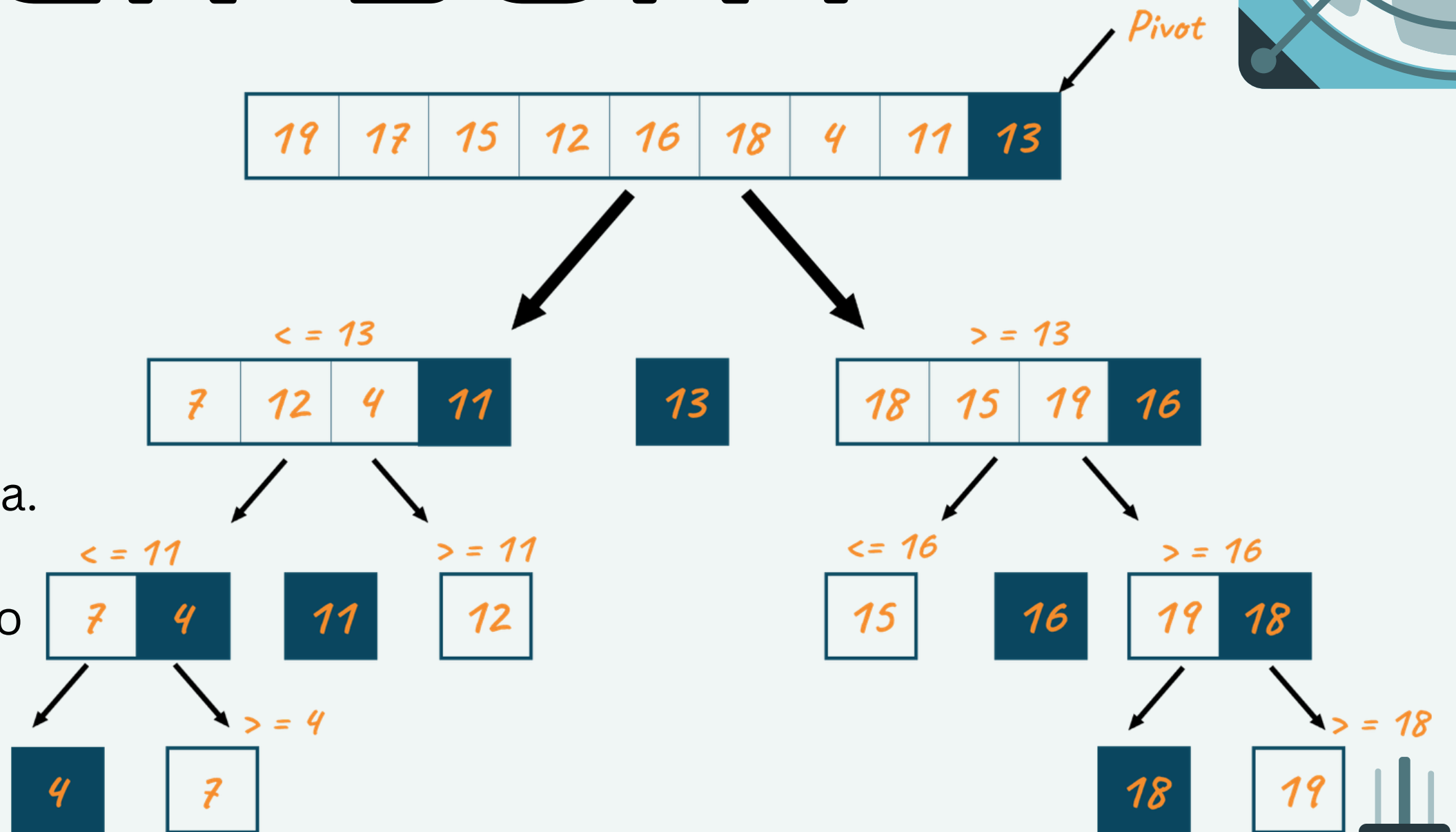


QUICK SORT

1. Pivote: Se selecciona cualquier número de la lista.

2. Divide: De tal manera que los menores al pivote están a su izquierda y los mayores a su derecha.

3. Recursividad: Se repite el proceso para la izquierda y la derecha.



LA COMPLEJIDAD TEMPORAL DEL ALGORITMO:



QUICK SORT (MEJOR CASO):

El pivote divide el arreglo en **dos mitades casi iguales** en cada partición (Pivote en el centro): $O(n \log n)$



QUICK SORT (PEOR CASO):

El pivote siempre es el **extremo pequeño** o el más **grande**, de modo que **una partición queda vacía** y la otra tiene todos los elementos menos uno (Pivote en el extremo): $O(n^2)$

MEJOR CASO

División Equitativa

3 2 8 | 5 | 9 7 6 4

5

Pivote en el centro

[3 2 4]

[8 9 7 6]

$O(n \log n)$

PEOR CASO

División Desbalanceada

2 | 3 8 5 9 7 6 4

Pivote en el extremo

[]

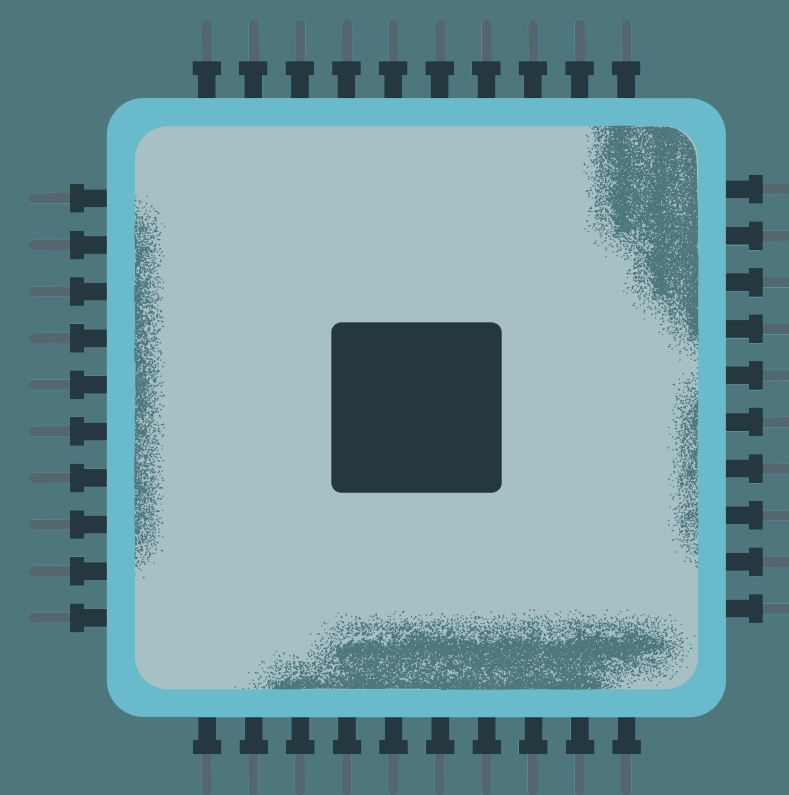
[3 8 5 9 7 6 4]

[2]

[3 8 5 9 7 6 4]

$O(n^2)$

MERGE SORT



1. DIVIDIR

Se divide el arreglo en dos mitades.

2. VENCER

Ordena recursivamente cada mitad.

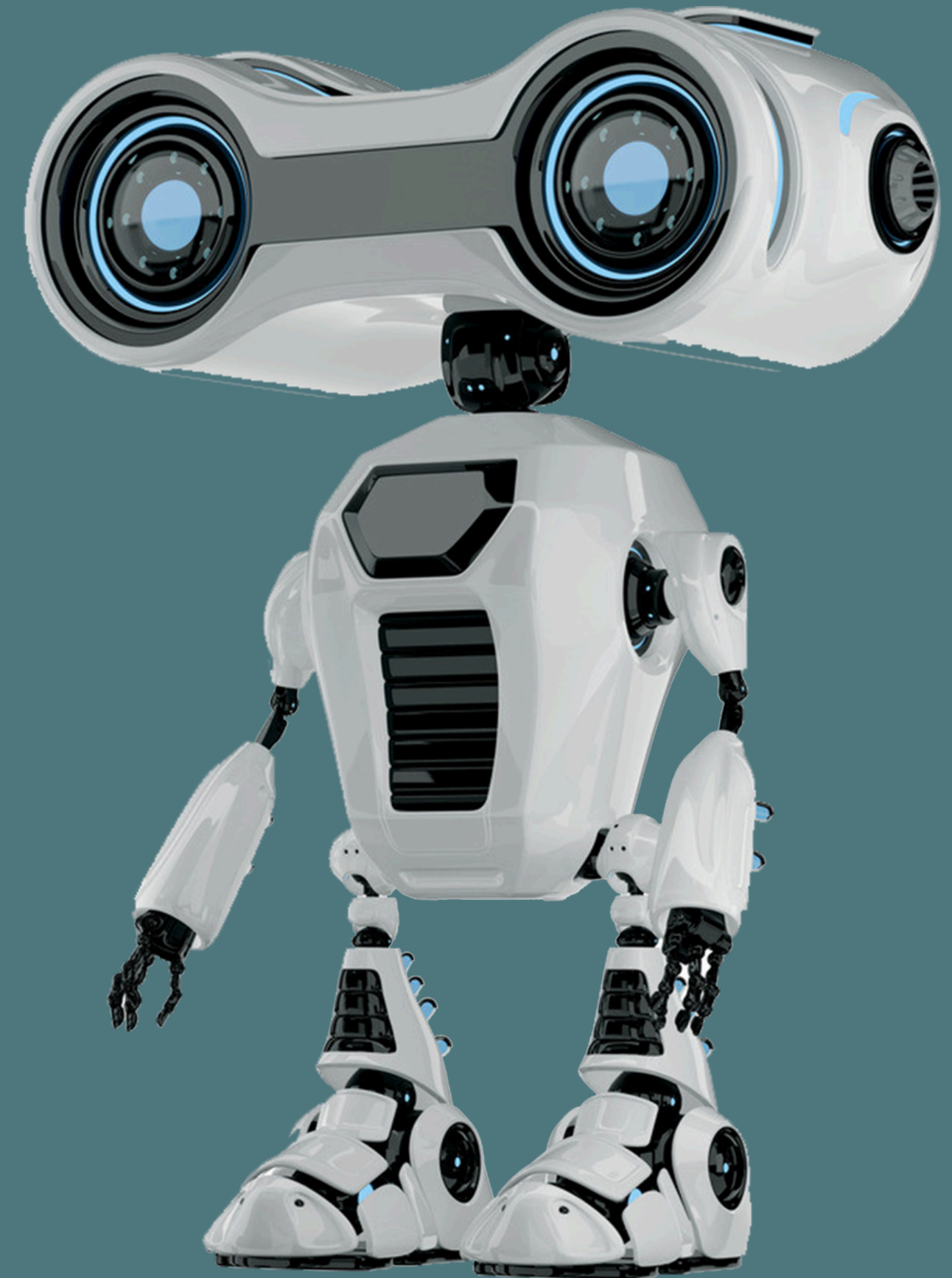
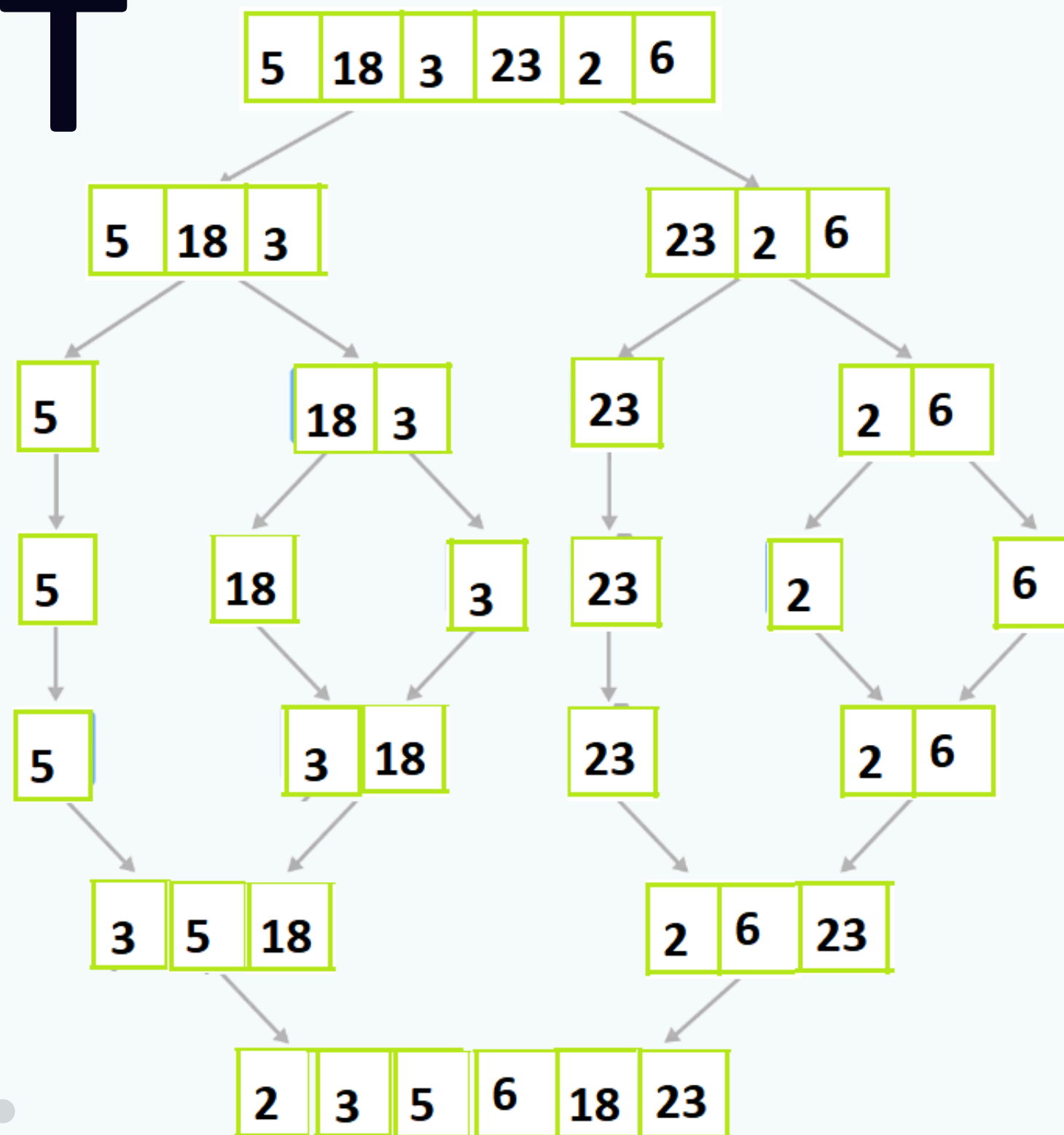
3. COMBINAR

Fusiona las mitades ya ordenadas.

COMPLEJIDAD TEMPORAL

$T(n) = O(n \log n)$, todos los casos posibles.

DIAGRAMMA MERGE SORT



ACTIVIDAD

P Q Q T T A E P M X E S S F Z C K C L U
X V L U R T Q E P J S G H F I R B B X K
R I P T O L K U J G A Z N L K O A R J Y
V G F V S A A M U L B A S E N Q F D X D
J L I G K K D O T R O S E G R E M H C I
O P U B C O K D E Q S B L O V R R N D A
I O O P I T L U O Z A N B B N E E R S Q
H U H H U N N L Q M C B A N U B Z M F W
Y S D F Q E A A C Y X D Z T E M K D B L
U P V R C I I R W D P S I G Y M I F C P
G L H D Y M D I Y N Z L L R E D G Q G N
R B V D Y A S D T S I W E J P O O Z B K
C W L V M N A A X Z E N L O G N X O M N
P K B I P E O D A Z W A A R V L K J E V
M R W M N D K C D U N Z R J V N J B S J
N B T Y N R I O T G O R A C M F Y I J H
N J S W S O F S M K P E P B H D M C M I
T T K C N M G L S P V T S J F A W T R C
F Q V X T O M V A G P K U J N R F L N X
V C Y B Z X T W U O S C F A P C H Q G C

1. Algoritmo de ordenamiento que usa la técnica divide y vencerás y selecciona un pivote para ordenar los elementos.
2. Algoritmo que permite encontrar un elemento en una colección ordenada, reduciendo el rango de búsqueda a la mitad en cada paso.
- 3.Cuál es la condición que debe cumplir una estructura para poder realizar una búsqueda binaria.
4. Característica que hace referencia a que los subproblemas independientes pueden resolverse en paralelo con múltiples hilos.
5. Capacidad de aplicar un algoritmo de divide y vencerás a problemas similares.
6. Dividir un problema complejo en subproblemas manejables.
7. Elemento que se elige en Quick Sort para dividir el arreglo en menores y mayores.
8. Algoritmo que divide la lista, ordena cada mitad recursivamente y luego combina los resultados.
9. Complejidad temporal del algoritmo Merge Sort en cualquier caso.
10. Característica que hace referencia a detener la recursión cuando se cumple cierta condición.

www.unsitiogenial.es

MUCHAS GRACIAS

